

Informe laboratorio 3

Grupo 123

Mariana Cediel 202321548

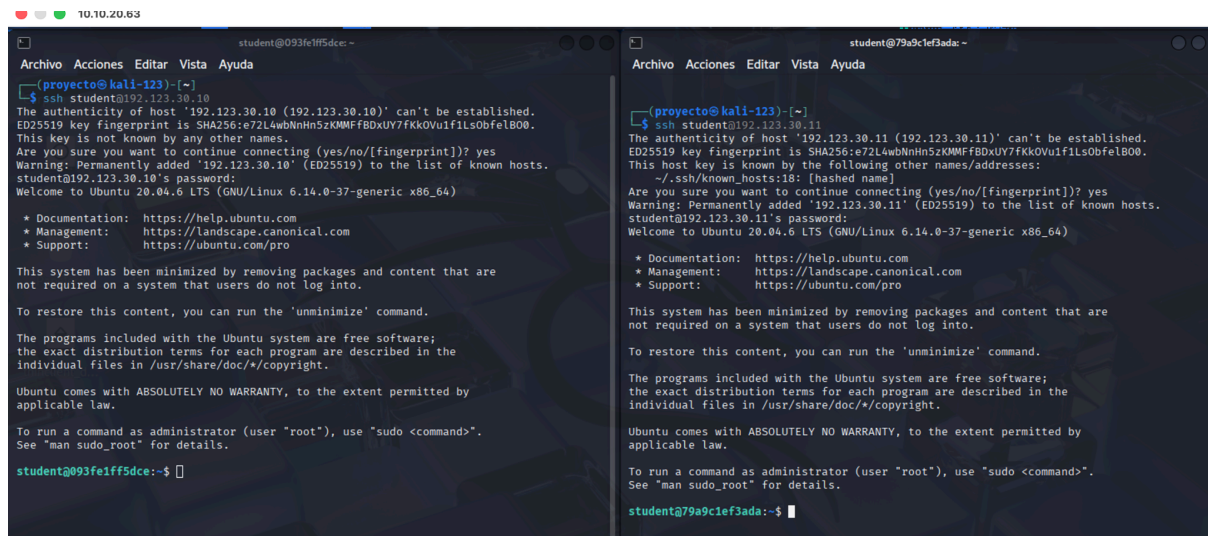
Valeria Martínez Vélez 202410228

Sofia Rodríguez Chaparro 202222588

Como equipo de consultoría en ciberseguridad, evaluamos dos mecanismos de distribución de claves simétricas para asegurar la comunicación entre estaciones de trabajo del segmento OTHER de la infraestructura: cifrado asimétrico (RSA) mediante un esquema de "sobre digital", y acuerdo de clave Diffie-Hellman (DH). Ambos esquemas se implementaron entre las máquinas student1 (IP OTHER.10) y student2 (IP OTHER.11), y se validó su correcto funcionamiento cifrando y descifrando archivos reales con AES.

1. Conexión SSH a las máquinas asignadas

Se estableció conexión SSH a ambas máquinas del segmento OTIC asignadas para el laboratorio, student1 (IP 192.123.30.10) y student2 (IP 192.123.30.11), utilizando las credenciales proporcionadas (usuario student). Al ser la primera conexión a cada host, se aceptó la verificación de autenticidad de la clave SSH (ED25519 key fingerprint) de cada servidor. Ambas conexiones se establecieron exitosamente sobre Ubuntu 20.04.6 LTS.



The image shows two terminal windows side-by-side. The left window is titled 'student@093fe1ff5dce: ~' and shows an SSH connection to 192.123.30.10. The right window is titled 'student@79a9c1ef3ada: ~' and shows an SSH connection to 192.123.30.11. Both windows display the same sequence of messages: a warning about the host's authenticity, a confirmation to add the fingerprint to the known hosts, a password prompt, and a welcome message to Ubuntu 20.04.6 LTS. The user 'student' is logged in on both machines.

```
student@093fe1ff5dce: ~  
Archivo Acciones Editar Vista Ayuda  
[proyecto@kali-123]-[~]  
$ ssh student@192.123.30.10  
The authenticity of host '192.123.30.10 (192.123.30.10)' can't be established.  
ED25519 key fingerprint is SHA256:e72L4wbNnHn5zKMMFF8DxUY7FkkOVuifLsObfel800.  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.123.30.10' (ED25519) to the list of known hosts.  
student@192.123.30.10's password:  
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.14.0-37-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
student@093fe1ff5dce:~$
```

```
student@79a9c1ef3ada: ~  
Archivo Acciones Editar Vista Ayuda  
[proyecto@kali-123]-[~]  
$ ssh student@192.123.30.11  
The authenticity of host '192.123.30.11 (192.123.30.11)' can't be established.  
ED25519 key fingerprint is SHA256:e72L4wbNnHn5zKMMFF8DxUY7FkkOVuifLsObfel800.  
This host key is known by the following other names/addresses:  
~/.ssh/known_hosts:18: [hashed name]  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.123.30.11' (ED25519) to the list of known hosts.  
student@192.123.30.11's password:  
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6.14.0-37-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
This system has been minimized by removing packages and content that are  
not required on a system that users do not log into.  
  
To restore this content, you can run the 'unminimize' command.  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
student@79a9c1ef3ada:~$
```

2. Cifrado asimétrico (RSA)

2.1 Generación de pares de claves

En cada máquina se generaron un par de claves RSA de 2048 bits mediante OpenSSL. Además, se verificó la creación exitosa de ambos archivos en cada máquina mediante ls -l.

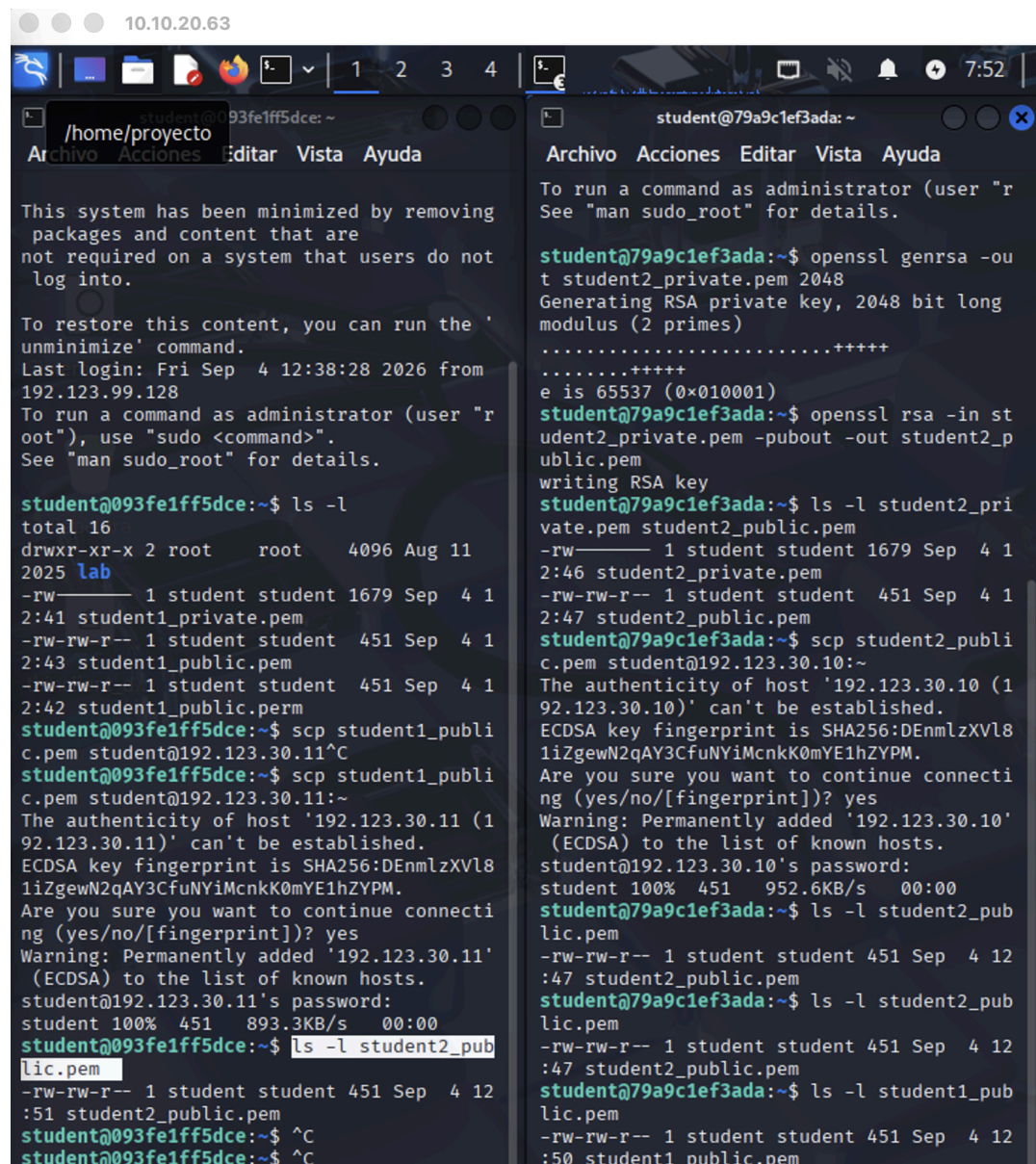
```
student@79a9c1ef3ada: ~  
Archivo Acciones Editar Vista Ayuda  
ED25519 key fingerprint is SHA256:e72L4wbN  
This host key is known by the following ot  
~/.ssh/known_hosts:18: [hashed name]  
Are you sure you want to continue connecti  
Warning: Permanently added '192.123.30.11'  
student@192.123.30.11's password:  
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 6  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canon  
* Support:       https://ubuntu.com/pro  
  
This system has been minimized by removing  
not required on a system that users do not  
  
To restore this content, you can run the '  
  
The programs included with the Ubuntu syst  
the exact distribution terms for each prog  
individual files in /usr/share/doc/*/copyr  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY,  
applicable law.  
  
To run a command as administrator (user "r  
See "man sudo_root" for details.  
  
student@79a9c1ef3ada:~$ openssl genrsa -ou  
t student2_private.pem 2048  
Generating RSA private key, 2048 bit long  
modulus (2 primes)  
.....+++++  
.....+++++  
e is 65537 (0x010001)  
student@79a9c1ef3ada:~$ openssl rsa -in st  
udent2_private.pem -pubout -out student2_p  
ublic.pem  
writing RSA key  
student@79a9c1ef3ada:~$ ls -l student2_pri  
vate.pem student2_public.pem  
-rw----- 1 student student 1679 Sep  4 1  
2:46 student2_private.pem  
-rw-rw-r-- 1 student student  451 Sep  4 1  
2:47 student2_public.pem  
student@79a9c1ef3ada:~$
```

2.2 Custodia de la clave privada

Las claves privadas se generaron con permisos restrictivos (-rw-----, equivalente a 600), visibles en la salida de ls -l, lo que garantiza que solo el usuario propietario puede leer el archivo. En ningún momento la clave privada de ninguna máquina fue transmitida por la red, únicamente permaneció en el sistema de archivos local de cada host.

2.3 Intercambio de la clave pública

Se intercambiaron únicamente las claves públicas entre las dos máquinas mediante scp:



```
10.10.20.63
/home/proyecto
Archivo Acciones Editar Vista Ayuda
This system has been minimized by removing packages and content that are not required on a system that users do not log into.
To restore this content, you can run the 'unminimize' command.
Last login: Fri Sep  4 12:38:28 2026 from 192.123.99.128
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
student@093fe1ff5dce:~$ ls -l
total 16
drwxr-xr-x 2 root root 4096 Aug 11 2025 lab
-rw-r--r-- 1 student student 1679 Sep  4 12:41 student1_private.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:43 student1_public.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:42 student1_public.perm
student@093fe1ff5dce:~$ scp student1_public.pem student@192.123.30.11^C
student@093fe1ff5dce:~$ scp student1_public.pem student@192.123.30.11:~
The authenticity of host '192.123.30.11 (192.123.30.11)' can't be established.
ECDSA key fingerprint is SHA256:DEnmlzXVl8liZgewN2qAY3CfuNYiMcnkK0mYE1hZYPm.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.123.30.11' (ECDSA) to the list of known hosts.
student@192.123.30.11's password:
student 100% 451 893.3KB/s 00:00
student@093fe1ff5dce:~$ ls -l student2_public.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:51 student2_public.pem
student@093fe1ff5dce:~$ ^C
student@093fe1ff5dce:~$ ^C

student@79a9c1ef3ada: ~
Archivo Acciones Editar Vista Ayuda
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
student@79a9c1ef3ada:~$ openssl genrsa -out student2_private.pem 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
student@79a9c1ef3ada:~$ openssl rsa -in student2_private.pem -pubout -out student2_public.pem
writing RSA key
student@79a9c1ef3ada:~$ ls -l student2_private.pem student2_public.pem
-rw-r--r-- 1 student student 1679 Sep  4 12:46 student2_private.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:47 student2_public.pem
student@79a9c1ef3ada:~$ scp student2_public.pem student@192.123.30.10:~
The authenticity of host '192.123.30.10 (192.123.30.10)' can't be established.
ECDSA key fingerprint is SHA256:DEnmlzXVl8liZgewN2qAY3CfuNYiMcnkK0mYE1hZYPm.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.123.30.10' (ECDSA) to the list of known hosts.
student@192.123.30.10's password:
student 100% 451 952.6KB/s 00:00
student@79a9c1ef3ada:~$ ls -l student2_public.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:47 student2_public.pem
student@79a9c1ef3ada:~$ ls -l student2_public.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:47 student2_public.pem
student@79a9c1ef3ada:~$ ls -l student1_public.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:50 student1_public.pem
```

2.4 Script Python para cifrado/descifrado asimétrico (sobre_digital.py)

Se implementó un script en Python (librería pycryptodome) que genera una clave simétrica AES-256 aleatoria y cifra esa clave usando la clave pública RSA del destinatario, con el esquema PKCS1_OAEP (padding recomendado para RSA, resistente a ataques de padding oracle). Además, permite al destinatario descifrar la clave AES usando su clave privada:

```
sudo apt update && sudo apt install python3-pip -y
pip3 install pycryptodome --break-system-packages
#!/usr/bin/env python3
"""
```

sobre_digital.py

Lab 3 - Cifrado Asimétrico y Distribucion de Claves

Genera una clave simétrica AES aleatoria y la cifra con RSA usando la

clave publica del destinatario. Tambien permite descifrarla con la clave privada del destinatario.

Uso:

```
# En el REMITENTE (genera una clave AES nueva y la cifra con RSA)
python3 sobre_digital.py generar_y_cifrar <clave_publica_destinatario.pem>
<clave_aes_cifrada.bin> <clave_aes_texto_plano.key>
```

```
# En el DESTINATARIO (descifra la clave AES con su clave privada)
python3 sobre_digital.py descifrar_clave <clave_privada_propia.pem>
<clave_aes_cifrada.bin> <clave_aes_recuperada.key>
"""
```

```
import sys
import os
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Random import get_random_bytes
```

```
def generar_y_cifrar(ruta_pub_destinatario: str, ruta_salida_cifrada: str,
ruta_salida_clave_plana: str):
    # 1. Generar una clave AES-256 aleatoria (32 bytes)
    clave_aes = get_random_bytes(32)

    # Guardamos la clave AES en texto plano (LOCAL, para usarla con cifrado_ftp.py)
    # En un caso real, este archivo se borraría despues de usarlo.
    with open(ruta_salida_clave_plana, "wb") as f:
        f.write(clave_aes)
    print(f"[OK] Clave AES generada y guardada temporalmente en:
{ruta_salida_clave_plana}")
    print(f"    (Esta clave en texto plano es SOLO para uso local del remitente)")

    # 2. Cargar la clave publica del destinatario
    with open(ruta_pub_destinatario, "rb") as f:
        clave_publica = RSA.import_key(f.read())

    # 3. Cifrar la clave AES usando RSA (PKCS1_OAEP es el esquema recomendado)
    cifrador_rsa = PKCS1_OAEP.new(clave_publica)
    clave_aes_cifrada = cifrador_rsa.encrypt(clave_aes)

    with open(ruta_salida_cifrada, "wb") as f:
        f.write(clave_aes_cifrada)

    print(f"[OK] Clave AES cifrada con RSA guardada en: {ruta_salida_cifrada}")
    print(f"    Tamano de la clave AES cifrada: {len(clave_aes_cifrada)} bytes")
```

```
def descifrar_clave(ruta_priv_propia: str, ruta_entrada_cifrada: str, ruta_salida_clave: str):
```

```
    # 1. Cargar la clave privada propia
    with open(ruta_priv_propia, "rb") as f:
        clave_privada = RSA.import_key(f.read())

    # 2. Leer la clave AES cifrada
    with open(ruta_entrada_cifrada, "rb") as f:
        clave_aes_cifrada = f.read()

    # 3. Descifrar usando RSA + la clave privada
    descifrador_rsa = PKCS1_OAEP.new(clave_privada)
    clave_aes = descifrador_rsa.decrypt(clave_aes_cifrada)

    with open(ruta_salida_clave, "wb") as f:
        f.write(clave_aes)

    print(f"[OK] Clave AES descifrada y guardada en: {ruta_salida_clave}")
```

```
def main():
```

```
    if len(sys.argv) != 5:
        print(__doc__)
        sys.exit(1)

    modo = sys.argv[1]

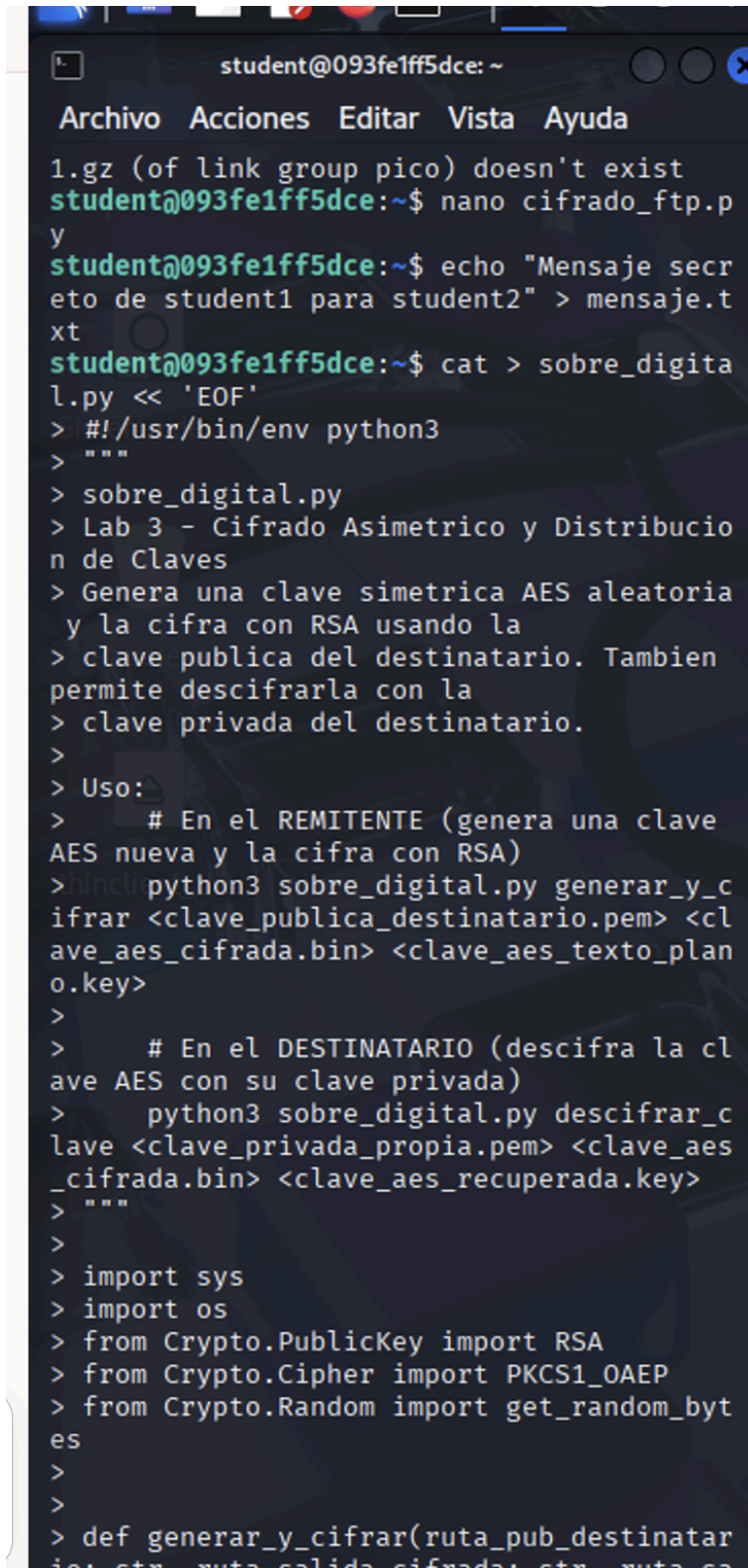
    if modo == "generar_y_cifrar":
        _, _, ruta_pub, ruta_salida_cifrada, ruta_salida_clave_plana = sys.argv
        if not os.path.isfile(ruta_pub):
            print(f"[ERROR] No existe: {ruta_pub}")
            sys.exit(1)
        generar_y_cifrar(ruta_pub, ruta_salida_cifrada, ruta_salida_clave_plana)

    elif modo == "descifrar_clave":
        _, _, ruta_priv, ruta_entrada_cifrada, ruta_salida_clave = sys.argv
        if not os.path.isfile(ruta_priv) or not os.path.isfile(ruta_entrada_cifrada):
            print("[ERROR] Alguno de los archivos de entrada no existe.")
            sys.exit(1)
        descifrar_clave(ruta_priv, ruta_entrada_cifrada, ruta_salida_clave)

    else:
        print("[ERROR] Modo no reconocido. Use 'generar_y_cifrar' o 'descifrar_clave'.")
        sys.exit(1)
```

```
if __name__ == "__main__":
    main()
```

Se creó el script sobre_digital.py en la máquina student1 mediante el comando `cat > sobre_digital.py << 'EOF'`, incorporando la lógica de generación de clave AES y cifrado con RSA descrita en la sección anterior.



```
student@093fe1ff5dce: ~  
Archivo Acciones Editar Vista Ayuda  
1.gz (of link group pico) doesn't exist  
student@093fe1ff5dce:~$ nano cifrado_ftp.p  
y  
student@093fe1ff5dce:~$ echo "Mensaje secr  
eto de student1 para student2" > mensaje.t  
xt  
student@093fe1ff5dce:~$ cat > sobre_digita  
l.py << 'EOF'  
> #!/usr/bin/env python3  
> """  
> sobre_digital.py  
> Lab 3 - Cifrado Asimetrico y Distribucio  
n de Claves  
> Genera una clave simetrica AES aleatoria  
y la cifra con RSA usando la  
> clave publica del destinatario. Tambien  
permite descifrarla con la  
> clave privada del destinatario.  
>  
> Uso:  
> # En el REMITENTE (genera una clave  
AES nueva y la cifra con RSA)  
> python3 sobre_digital.py generar_y_c  
ifrar <clave_publica_destinatario.pem> <cl  
ave_aes_cifrada.bin> <clave_aes_texto_plan  
o.key>  
>  
> # En el DESTINATARIO (descifra la cl  
ave AES con su clave privada)  
> python3 sobre_digital.py descifrar_c  
lave <clave_privada_propia.pem> <clave_aes  
_cifrada.bin> <clave_aes_recuperada.key>  
> """  
>  
> import sys  
> import os  
> from Crypto.PublicKey import RSA  
> from Crypto.Cipher import PKCS1_OAEP  
> from Crypto.Random import get_random_byt  
es  
>  
>  
> def generar_y_cifrar(ruta_pub_destinatar  
io: str, ruta_salida_cifrada: str, ruta_sa
```

Se ejecutó el comando python3 sobre_digital.py generar_y_cifrar student2_public.pem clave_aes_cifrada.bin clave_aes.key, obteniendo como resultado la generación de una clave AES-256 aleatoria y su cifrado exitoso utilizando la clave pública de student2, produciendo un archivo de 256 bytes (tamaño esperado para una clave cifrada con RSA-2048).

```
if __name__ == '__main__':
    main()
student@093fe1ff5dce:~$ python3 sobre_digital.py generar_y_cifrar student2_public.pem clave_aes_cifrada.bin clave_aes.key
[OK] Clave AES generada y guardada temporalmente en: clave_aes.key
      (Esta clave en texto plano es SOLO para uso local del remitente)
[OK] Clave AES cifrada con RSA guardada en: clave_aes_cifrada.bin
      Tamano de la clave AES cifrada: 256 bytes
student@093fe1ff5dce:~$
```

Se recreó el script cifrado_ftp.py (desarrollado originalmente en el Lab 2) en la máquina student1, verificando su correcta sintaxis mediante python3 -m py_compile cifrado_ftp.py antes de su uso.

```

Archivo Acciones Editar Vista Ayuda
> print(f"[OK] Archivo descifrado guardado en: {ruta_salida}")
>
>
> def main():
>     if len(sys.argv) != 4:
>         print(__doc__)
>         sys.exit(1)
>
>     modo, entrada, salida = sys.argv[1], sys.argv[2], sys.argv[3]
>
>     if not os.path.isfile(entrada):
>         print(f"[ERROR] El archivo de entrada no existe: {entrada}")
>         sys.exit(1)
>
>     password = getpass.getpass("Ingrese la contraseña de cifrado: ")
>
>     if modo == "cifrar":
>         cifrar_archivo(entrada, salida, password)
>     elif modo == "descifrar":
>         descifrar_archivo(entrada, salida, password)
>     else:
>         print("[ERROR] Modo no reconocido. Use 'cifrar' o 'descifrar'.")
>         sys.exit(1)
>
>
> if __name__ == "__main__":
>     main()
> EOF
student@093fe1ff5dce:~$ python3 -m py_compile cifrado_ftp.py
student@093fe1ff5dce:~$ echo "Mensaje secreto de student1 para student2" > mensaje.txt
student@093fe1ff5dce:~$ base64 clave_aes.key
vmvtW2r85RQgvDmWr6t5G0hYaqE0Ie8NsoELKUIg0lS=

```

2.5 Cifrado del mensaje con la clave AES generada por RSA

Se generó un archivo de prueba (mensaje.txt) y se obtuvo la representación en base64 de la clave AES (clave_aes.key) para utilizarla como contraseña de entrada del script cifrado_ftp.py. Se ejecutó el cifrado (python3 cifrado_ftp.py cifrar mensaje.txt mensaje.txt.enc), y se verificó mediante cat mensaje.txt.enc que el contenido resultante es ilegible, confirmando el cifrado exitoso del archivo con la clave simétrica protegida por RSA.

```

student@093fe1ff5dce:~$ base64 clave_aes.k
ey
vmvtW2r85RQgvDmWr6t5G0hYaqE0Ie8NsoELKUIg0l
s=
student@093fe1ff5dce:~$ python3 cifrado_ft
p.py cifrar mensaje.txt mensaje.txt.enc
Ingrese la contraseña de cifrado:
[OK] Archivo cifrado guardado en: mensaje.
txt.enc
student@093fe1ff5dce:~$ cat mensaje.txt.en
c
♦♦PR♦
S♦♦~♦♦♦E♦8♦♦"T#♦uL♦♦♦♦H♦♦F; ♦♦♦Ku♦♦♦y♦T♦♦
gk=,♦rqd"♦♦♦♦5♦♦♦k}i-0♦student@093fe1ff5d
ce:~$

```

Tras completar el flujo del sobre digital (envío del archivo cifrado y la clave AES cifrada con RSA hacia student2, descifrado de la clave con la clave privada de student2, y descifrado del archivo con la clave AES recuperada), se verificó mediante `ls -lh` en student2 la presencia de todos los archivos generados durante el proceso (cifrado_ftp.py, clave_aes_cifrada.bin, mensaje.txt.enc, sobre_digital.py, y las claves públicas/privadas).

```

-rw-rw-r-- 1 student student 1.7K Sep  4 12:46 student2_private.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:47 student2_public.pem
student@79a9c1ef3ada:~$ ls -lh
total 32K
-rw-rw-r-- 1 student student 2.6K Sep  4 15:39 cifrado_ftp.py
-rw-rw-r-- 1 student student 256 Sep  4 15:39 clave_aes_cifrada.bin
drwxr-xr-x 2 root root 4.0K Aug 11 2025 lab
-rw-rw-r-- 1 student student 80 Sep  4 15:39 mensaje.txt.enc
-rw-rw-r-- 1 student student 3.6K Sep  4 15:39 sobre_digital.py
-rw-rw-r-- 1 student student 451 Sep  4 12:50 student1_public.pem
-rw-rw-r-- 1 student student 1.7K Sep  4 12:46 student2_private.pem
-rw-rw-r-- 1 student student 451 Sep  4 12:47 student2_public.pem
student@79a9c1ef3ada:~$

```

Se confirmó que el contenido de `mensaje_recuperado.txt` en student2 es idéntico al de `mensaje.txt` original en student1, validando el éxito completo del esquema RSA de sobre digital.

```

student@093fe1ff5dce:~$ ls
_pycache      lab          student1_private.pem  cifrado_ftp.py  mensaje.txt.enc  student2_private.pem
cifrado_ftp.py  mensaje.txt  student1_public.pem   clave_aes_cifrada.bin  sobre_digital.py  student2_public.pem
clave_aes.key   mensaje.txt.enc  student1_public.pem  clave_aes_recuperada.key  sobre_digital.py  student1_public.pem
lab            sobre_digital.py  student2_public.pem
student@093fe1ff5dce:~$ cat mensaje.txt
Mensaje secreto de student1 para student2
student@093fe1ff5dce:~$ sha256sum mensaje.txt
66e70e98a5ece49f067d1f2d7a73067573c5e26d110a0c994b6e15747c26e761  mensaje.tx
t
student@093fe1ff5dce:~$
student@79a9c1ef3ada:~$ cat mensaje_recuperado.txt
Mensaje secreto de student1 para student2
student@79a9c1ef3ada:~$ sha256sum mensaje_recuperado.txt
66e70e98a5ece49f067d1f2d7a73067573c5e26d110a0c994b6e15747c26e761  mensaje_rec
uperado.txt
student@79a9c1ef3ada:~$

```

3. Distribución de claves simétricas (Diffie-Hellman) con canal autenticado

3.1 Instalación de la librería cryptography

Dado que pycryptomode no incluye una implementación nativa de Diffie-Hellman, se instaló la librería cryptography en ambas máquinas.

```
student@093fe1ff5dce:~$ pip install --user cryptography
Collecting cryptography
  Downloading cryptography-47.0.0-cp38-abi3-manylinux2014_x86_64.manylinux2
  _17_x86_64.whl (4.7 MB)
    | 4.7 MB 9.6 MB/s
Collecting typing-extensions>=4.13.2; python_full_version < "3.11"
  Downloading typing_extensions-4.13.2-py3-none-any.whl (45 kB)
    | 45 kB 804 kB/s
Collecting cffi>=1.14; python_full_version == "3.8.*" and platform_python_im
  plementation != "PyPy"
  Downloading cffi-1.17.1-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_
  64.whl (446 kB)
    | 446 kB 11.8 MB/s
Collecting pycparser
  Downloading pycparser-2.23-py3-none-any.whl (118 kB)
    | 118 kB 11.0 MB/s
Installing collected packages: typing-extensions, pycparser, cffi, cryptogra
  phy
Successfully installed cffi-1.17.1 cryptography-47.0.0 pycparser-2.23 typing
  -extensions-4.13.2

student@79a9c1ef3ada:~$ pip install --user cryptography
Collecting cryptography
  Downloading cryptography-47.0.0-cp38-abi3-manylinux2014_x86_64.manylinux_2
  _17_x86_64.whl (4.7 MB)
    | 4.7 MB 11.1 MB/s
Collecting typing-extensions>=4.13.2; python_full_version < "3.11"
  Downloading typing_extensions-4.13.2-py3-none-any.whl (45 kB)
    | 45 kB 378 kB/s
Collecting cffi>=1.14; python_full_version == "3.8.*" and platform_python_imp
  lementation != "PyPy"
  Downloading cffi-1.17.1-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_6
  4.whl (446 kB)
    | 446 kB 11.6 MB/s
Collecting pycparser
  Downloading pycparser-2.23-py3-none-any.whl (118 kB)
    | 118 kB 10.5 MB/s
Installing collected packages: typing-extensions, pycparser, cffi, cryptograp
  hy
Successfully installed cffi-1.17.1 cryptography-47.0.0 pycparser-2.23 typing-
  extensions-4.13.2
```

3.2 Generación de parámetros públicos

Student1 generó los parámetros públicos del grupo Diffie-Hellman (módulo primo p de 2048 bits y generador $g=2$), que no son secretos y por tanto pueden transmitirse en claro. para esto se creó un archivo llamado generar_parametric_dh.py con el siguiente script:

```
student@093fe1ff5dce: ~
Archivo Acciones Editar Vista Ayuda
GNU nano 4.8 generar_parametros_dh.py Modified
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization

parameters = dh.generate_parameters(generator = 2, key_size = 2048)
pem = parameters.parameter_bytes(
    encoding = serialization.Encoding.PEM,
    format = serialization.ParameterFormat.PKCS3
)

with open("dh_parametros.pem", "wb") as f:
    f.write(pem)

print("[OK] Parámetros DH generados en dh_parametros.pem")
```

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization
```

```
parameters = dh.generate_parameters(generator=2, key_size=2048)
```

```
pem = parameters.parameter_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.ParameterFormat.PKCS3
)
```

```
with open("dh_parametros.pem", "wb") as f:
    f.write(pem)
```

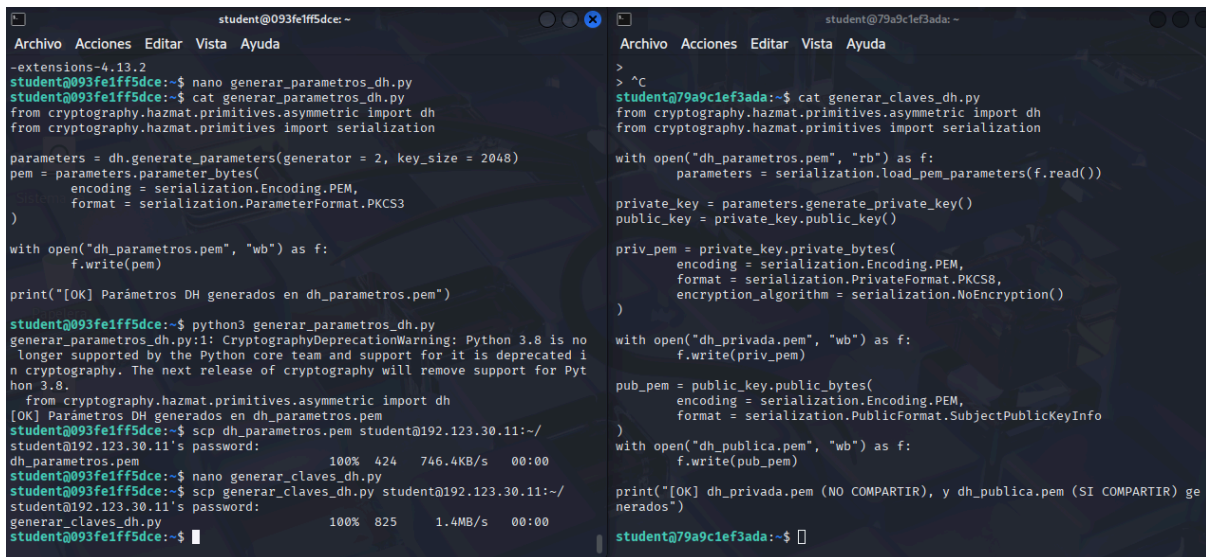
```
print("[OK] Parámetros DH generados en dh_parametros.pem")
```

Se envían los parámetros públicos a student2:

```
[OK] Parámetros DH generados en dh_parametros.pem
student@093fe1ff5dce:~$ scp dh_parametros.pem student@192.123.30.11:~/
student@192.123.30.11's password:
dh_parametros.pem                                100% 424   746.4KB/s   00:00
student@093fe1ff5dce:~$
```

3.3 Generación del par de claves privada/pública DH en cada máquina

En ambas máquinas se ejecutó el mismo script (generar_claves_dh.py), que a partir de los parámetros compartidos genera un par de claves propio: una privada (dh_privada.pem, que nunca se comparte) y una pública (dh_publica.pem, que sí se transmite).



```
student@093fe1ff5dce:~$ nano generar_parametros_dh.py
-extensions~4.13.2
student@093fe1ff5dce:~$ cat generar_parametros_dh.py
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization

parameters = dh.generate_parameters(generator = 2, key_size = 2048)
pem = parameters.parameter_bytes(
    encoding = serialization.Encoding.PEM,
    format = serialization.PrivateFormat.PKCS3
)

with open("dh_parametros.pem", "wb") as f:
    f.write(pem)

print("[OK] Parámetros DH generados en dh_parametros.pem")

student@093fe1ff5dce:~$ python3 generar_parametros_dh.py
generar_parametros_dh.py:1: CryptographyDeprecationWarning: Python 3.8 is no
longer supported by the Python core team and support for it is deprecated i
n cryptography. The next release of cryptography will remove support for Pyt
hon 3.8.
[OK] Parámetros DH generados en dh_parametros.pem
student@093fe1ff5dce:~$ scp dh_parametros.pem student@192.123.30.11:~/
student@192.123.30.11's password:
dh_parametros.pem                                100% 424   746.4KB/s   00:00
student@093fe1ff5dce:~$ nano generar_claves_dh.py
student@093fe1ff5dce:~$ scp generar_claves_dh.py student@192.123.30.11:~/
student@192.123.30.11's password:
generar_claves_dh.py                             100% 825   1.4MB/s     00:00
student@093fe1ff5dce:~$

student@79a9c1ef3ada:~$ cat generar_claves_dh.py
> ^C
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization

with open("dh_parametros.pem", "rb") as f:
    parameters = serialization.load_pem_parameters(f.read())

private_key = parameters.generate_private_key()
public_key = private_key.public_key()

priv_pem = private_key.private_bytes(
    encoding = serialization.Encoding.PEM,
    format = serialization.PrivateFormat.PKCS8,
    encryption_algorithm = serialization.NoEncryption()
)

with open("dh_privada.pem", "wb") as f:
    f.write(priv_pem)

pub_pem = public_key.public_bytes(
    encoding = serialization.Encoding.PEM,
    format = serialization.PublicFormat.SubjectPublicKeyInfo
)

with open("dh_publica.pem", "wb") as f:
    f.write(pub_pem)

print("[OK] dh_privada.pem (NO COMPARTIR), y dh_publica.pem (SI COMPARTIR) ge
nerados")

student@79a9c1ef3ada:~$
```

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization
```

```
with open("dh_parametros.pem", "rb") as f:
    parameters = serialization.load_pem_parameters(f.read())
```

```
private_key = parameters.generate_private_key()
public_key = private_key.public_key()
```

```
priv_pem = private_key.private_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PrivateFormat.PKCS8,
    encryption_algorithm=serialization.NoEncryption()
)
```

```
with open("dh_privada.pem", "wb") as f:
    f.write(priv_pem)
```

```
pub_pem = public_key.public_bytes(
    encoding=serialization.Encoding.PEM,
    format=serialization.PublicFormat.SubjectPublicKeyInfo
)
```

```
with open("dh_publica.pem", "wb") as f:
    f.write(pub_pem)
```

```
print("[OK] dh_privada.pem (NO compartir) y dh_publica.pem (sí compartir)
generados")
```

Se ejecuta el archivo generar_claves_dh.py en cada máquina:

```
generar_claves_dh.py
student@093fe1ff5dce:~$ python3 generar_claves_dh.py
generar_claves_dh.py:1: CryptographyDeprecationWarning: Python 3.8 is no longer supported by the Python core team and support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.8.
from cryptography.hazmat.primitives.asymmetric import dh
[OK] dh_privada.pem (NO COMPARTIR), y dh_publica.pem (SI COMPARTIR) generados
student@093fe1ff5dce:~$ █

student@79a9c1ef3ada:~$ python3 generar_claves_dh.py
generar_claves_dh.py:1: CryptographyDeprecationWarning: Python 3.8 is no longer supported by the Python core team and support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.8.
from cryptography.hazmat.primitives.asymmetric import dh
[OK] dh_privada.pem (NO COMPARTIR), y dh_publica.pem (SI COMPARTIR) generados
student@79a9c1ef3ada:~$ █
```

3.4 Intercambio de claves públicas DH

Cada máquina envió únicamente su clave pública DH a la otra. Las claves privadas permanecieron siempre en su máquina de origen:

```
192.123.30.10
student@093fe1ff5dce:~$ scp dh_publica.pem student@192.123.30.11:~/student1_
dh_publica.pem
student@192.123.30.11's password:
dh_publica.pem
100% 800 1.3MB/s 00:00
student@093fe1ff5dce:~$ █

192.123.30.11
student@79a9c1ef3ada:~$ scp dh_publica.pem student@192.123.30.10:~/student2_d
h_publica.pem
student@192.123.30.10's password:
dh_publica.pem
100% 800 2.0MB/s 00:00
student@79a9c1ef3ada:~$ █
```

3.5 Cálculo de la clave compartida

Se creó el script calcular_clave_compartida.py, que carga la clave privada propia y la pública recibida del otro extremo, ejecuta el intercambio Diffie-Hellman (exchange) para obtener el secreto compartido, y deriva de él una clave AES de 256 bits mediante HKDF-SHA256 (usando el mismo parámetro info en ambas máquinas para garantizar que la derivación sea idéntica).

```
import sys
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
```

```
peer_pub_file = sys.argv[1]
```

```
with open("dh_privada.pem", "rb") as f:
    private_key = serialization.load_pem_private_key(f.read(), password=None)
```

```
with open(peer_pub_file, "rb") as f:
    peer_public_key = serialization.load_pem_public_key(f.read())
```

```
shared_secret = private_key.exchange(peer_public_key)
```

```
derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=None,
    info=b"canal-dh-student1-student2"
).derive(shared_secret)
```

```

with open("clave_aes_dh.key", "wb") as f:
    f.write(derived_key)

print("[OK] Clave compartida derivada en clave_aes_dh.key")

```

El script se creó en Student1 y se le envió a Student2:
student1:

```

student@093fe1ff5dce:~$ nano calcular_clave_compartida.py
student@093fe1ff5dce:~$ scp calcular_clave_compartida.py student@192.123.30.
11:~/
student@192.123.30.11's password:
calcular_clave_compartida.py          100% 710      1.4MB/s   00:00
student@093fe1ff5dce:~$ █

```

student2:

```

student@79a9c1ef3ada:~$ cat calcular_clave_compartida.py
import sys
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

peer_pub_file = sys.argv[1]

with open("dh_privada.pem", "rb") as f:
    private_key = serialization.load_pem_private_key(f.read(), password=None)

with open(peer_pub_file, "rb") as f:
    peer_public_key = serialization.load_pem_public_key(f.read())

shared_secret = private_key.exchange(peer_public_key)

derived_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=None,
    info=b"canal-dh-student1-student2"
).derive(shared_secret)

with open("clave_aes_dh.key", "wb") as f:
    f.write(derived_key)

print("[OK] Clave compartida derivada en clave_aes_dh.key")
student@79a9c1ef3ada:~$ █

```

Se ejecuta el archivo calcular_clave_compartida.py para que cada máquina calcule la clave compartida:

```

student@093fe1ff5dce:~$ python3 calcular_clave_compartida.py student2_dh_publica.pem
calcular_clave_compartida.py:2: CryptographyDeprecationWarning: Python 3.8 is no longer supported by the Python core team and support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.8.
  from cryptography.hazmat.primitives import serialization, hashes
[OK] Clave compartida derivada en clave_aes_dh.key
student@093fe1ff5dce:~$ █

student@79a9c1ef3ada:~$ python3 calcular_clave_compartida.py student1_dh_publica.pem
calcular_clave_compartida.py:2: CryptographyDeprecationWarning: Python 3.8 is no longer supported by the Python core team and support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.8.
  from cryptography.hazmat.primitives import serialization, hashes
[OK] Clave compartida derivada en clave_aes_dh.key
student@79a9c1ef3ada:~$ █

```

3.6 Prueba de que ambas máquinas llegaron a la misma clave

Antes de utilizar la clave derivada, se verificó que ambas máquinas llegaron efectivamente al mismo valor, comparando el hash SHA-256 del archivo clave_aes_dh.key en cada extremo. El acuerdo Diffie-Hellman funcionó, sin que la clave viajara nunca en claro por la red. Lo verificamos porque los hashes coinciden perfectamente:

En student1:

```
student@093fe1ff5dce:~$ sha256sum clave_aes_dh.key
5eba75f15509979078b8d3b0bbc9203fbf05dd570fb96c8c34da7f082e63327a  clave_aes_dh.key
student@093fe1ff5dce:~$
```

En student2:

```
student@79a9c1ef3ada:~$ sha256sum clave_aes_dh.key
5eba75f15509979078b8d3b0bbc9203fbf05dd570fb96c8c34da7f082e63327a  clave_aes_dh.key
student@79a9c1ef3ada:~$
```

3.7 Cifrado y descifrado de un archivo con la clave DH

Finalmente, se probó el uso práctico de la clave acordada, cifrando y descifrando un archivo con AES (script cifrado_ftp.py) entre las dos máquinas.

Se creó un archivo de prueba mensaje_dh.txt en student 1:

```
student@093fe1ff5dce:~$ echo "Mensaje secreto usando Diffie-Hellman" > mensaje_dh.txt
student@093fe1ff5dce:~$ cat mensaje_dh.txt
Mensaje secreto usando Diffie-Hellman
student@093fe1ff5dce:~$
```

Se obtuvo la clave DH en base64 desde student1 usada como contraseña de cifrado:

```
student@093fe1ff5dce:~$ base64 clave_aes_dh.key
Y9OS20zLixQApQDho3/QdnKFxa1i2SwWzxtPy1Lq1ao=
student@093fe1ff5dce:~$
```

Se cifró el archivo con AES usando la clave DH base64 creada en el paso anterior:

```
student@093fe1ff5dce:~$ python3 cifrado_ftp.py cifrar mensaje_dh.txt mensaje_dh.txt.enc
Ingrese la contraseña de cifrado:
[OK] Archivo cifrado guardado en: mensaje_dh.txt.enc
student@093fe1ff5dce:~$
```

Se verifica la creación del archivo cifrado mensaje_dh.txt.enc:

```
[OK] Archivo cifrado guardado en: mensaje_dh.txt.enc
student@093fe1ff5dce:~$ ls -lh mensaje_dh.txt.enc
-rw-rw-r-- 1 student student 80 Sep  5 16:42 mensaje_dh.txt.enc
student@093fe1ff5dce:~$
```

Se envió el archivo cifrado a student2:

```
student@093fe1ff5dce:~$ scp mensaje_dh.txt.enc student@192.123.30.11:~/
student@192.123.30.11's password:
mensaje_dh.txt.enc                                100% 80 144.5KB/s 00:00
student@093fe1ff5dce:~$
```

Se obtuvo la clave DH en base64 en student2 (idéntica a la de student1):

```
student@79a9c1ef3ada:~$ base64 clave_aes_dh.key
Y90S20zLixQApQDho3/QdnKFxa1i2SwWzxtPy1Lq1ao=
student@79a9c1ef3ada:~$
```

Se descifró el archivo en student2 utilizando la clave derivada por Diffie-Hellman:

```
Y90S20zLixQApQDho3/QdnKFxa1i2SwWzxtPy1Lq1ao=
student@79a9c1ef3ada:~$ python3 cifrado_ftp.py descifrar mensaje_dh.txt.enc m
ensaje_dh_recuperado.txt
Ingrese la contraseña de cifrado:
[OK] Archivo descifrado guardado en: mensaje_dh_recuperado.txt
student@79a9c1ef3ada:~$
```

3.8 Verificación de integridad

Se comparó nuevamente el hash SHA-256 del archivo original y del archivo recuperado:

Desde student1:

```
student@093fe1ff5dce:~$ sha256sum mensaje_dh.txt
04457c66e5a1d6e8209cc2d4a7457b90110065bec0ab68db1a784122a75ee784 mensaje_dh
.txt
student@093fe1ff5dce:~$
```

Desde student2:

```
[OK] Archivo descifrado guardado en: mensaje_dh_recuperado.txt
student@79a9c1ef3ada:~$ cat mensaje_dh_recuperado.txt
Mensaje secreto usando Diffie-Hellman
student@79a9c1ef3ada:~$ sha256sum mensaje_dh_recuperado.txt
04457c66e5a1d6e8209cc2d4a7457b90110065bec0ab68db1a784122a75ee784 mensaje_dh_
recuperado.txt
student@79a9c1ef3ada:~$
```

Los hashes coinciden perfectamente. Se acordó una clave simétrica sin transmitirla directamente, solo viajaron los parámetros públicos y las claves públicas DH, y esa clave cifró y descifró un archivo correctamente con AES, confirmando el éxito integral del esquema Diffie-Hellman implementado.

4. Análisis comparativo

4.1 Cifrado simétrico vs. cifrado asimétrico

Ambos esquemas implementados en este laboratorio (RSA y Diffie-Hellman) tienen un propósito común: distribuir de forma segura una clave que después se usa con cifrado simétrico (AES) para proteger los datos. Esto no es casualidad: en la práctica, el cifrado asimétrico casi nunca se usa para cifrar directamente grandes volúmenes de información, por las siguientes razones.

ASPECTO	CIFRADO SIMÉTRICO (AES)	CIFRADO ASIMÉTRICO (RSA)
Rendimiento	Muy alto. Opera sobre bloques de bytes y se beneficia de aceleración por hardware. Adecuado para cifrar archivos y flujos grandes.	Órdenes de magnitud más lenta. Involucra exponenciación modular con enteros muy grandes. Impráctico para cifrar directamente volúmenes grandes de datos.
Límite de tamaño del mensaje	Puede cifrar archivos de cualquier tamaño en modo de bloque o flujo.	Limitado por el tamaño de la llave (con RSA-2048 y OAEP, unos 245 bytes por operación). Por eso solo se usa para envolver una clave AES, no el archivo completo.
Escalabilidad de claves	Para N partes que necesiten canales privados entre sí se requieren hasta $N(N-1)/2$ claves compartidas distintas.	Cada parte necesita un solo par de llaves. Solo se distribuyen las públicas, sin necesidad de secretos compartidos por adelantado.
Distribución de claves	Requiere un canal seguro previo o fuera de banda para compartir la clave. Si se transmite en claro por un canal no autenticado, es vulnerable a interceptación.	La llave pública puede distribuirse por canales no confidenciales; solo la llave privada requiere custodia estricta.

4.2 RSA vs. Diffie-Hellman

Aunque ambos esquemas se usan para "distribución de claves", resuelven problemas distintos y no son sustitutos directos entre sí.

ASPECTO	RSA	DIFFIE-HELLMAN
Objetivo primario	Cifrar/transportar un secreto hacia una identidad específica, usando su llave pública.	Acordar un secreto compartido entre dos partes sin transmitirlo nunca por el canal.
Base matemática	Dificultad de factorizar el producto de dos números primos grandes.	Dificultad del logaritmo discreto en un grupo cíclico.
Autenticación	Puede dar autenticación/no repudio si se usa para firmar. Requiere PKI o distribución confiable de la llave pública.	No autentica por sí mismo. Es vulnerable a man-in-the-middle si no se combina con firmas o certificados (DH autenticado).
Perfect Forward Secrecy	No. Si la llave privada se compromete, se pueden descifrar sesiones pasadas grabadas.	Sí, si se usan exponentes efímeros (DHE/ECDHE): cada sesión genera un secreto distinto.
Costo computacional	Alto para cifrar directamente datos grandes, se usa para proteger una clave simétrica	Moderado, se ejecuta una vez por sesión para derivar la clave, no para cifrar los datos.

	pequeña (sobre digital).	
Qué viaja por la red	La llave pública del destinatario y la clave AES cifrada con esa llave.	Los parámetros públicos (p, g) y las claves públicas DH de cada parte, nunca el secreto compartido.
Riesgo principal	Compromiso de la llave privada. Ataques si el módulo o el padding son débiles.	Ataque de intermediario si los parámetros o las públicas no se autentican al intercambiarlas.

4.3 Riesgos y beneficios observados en la implementación

	Beneficios	Riesgos
RSA	Permite dirigir el secreto a una identidad específica usando solo su llave pública, No requiere que las partes coordinen nada por adelantado más que el intercambio de públicas. Es la base de la autenticación en PKI/certificados.	La llave privada es un punto único de fallo, su robo compromete todo el tráfico pasado y futuro cifrado hacia esa identidad (no hay Perfect Forward Secrecy). El intercambio de llaves públicas debe autenticarse (por ejemplo con certificados) para evitar que un atacante sustituya la llave pública del destinatario (ataque MitM).
Diffie - Hellman	Las claves privadas usadas son efímeras y pueden descartarse tras derivar el secreto, lo que brinda Perfect Forward Secrecy si se generan nuevos pares en cada sesión. No es necesario poseer de antemano la llave pública "del otro" en el sentido de cifrado dirigido, solo acordar parámetros comunes.	Por sí solo no autentica a las partes: un atacante en el camino podría interceptar el intercambio de públicas y establecer dos secretos distintos, uno con cada parte. En este laboratorio ese riesgo se mitigó parcialmente al transmitir las claves públicas por SCP sobre un canal SSH autenticado con contraseña, pero en un entorno real se recomienda combinar DH con firmas digitales o certificados.

5. Recomendación final para la organización

Con base en las pruebas realizadas, se recomienda a la organización adoptar un modelo híbrido para asegurar la comunicación entre las estaciones de trabajo y los microservicios de la nueva plataforma analítica, en lugar de elegir un único esquema de forma exclusiva:

- Usar un esquema basado en RSA (o, preferiblemente, certificados X.509 con un esquema de firma como ECDSA) para autenticar la identidad de cada nodo o servicio, de manera análoga a como un servidor TLS presenta su certificado.

- Usar Diffie-Hellman efímero (idealmente su variante sobre curvas elípticas, ECDHE) para acordar, en cada sesión, una clave simétrica distinta, obteniendo Perfect Forward Secrecy — de modo que si una llave privada de largo plazo se compromete en el futuro, el tráfico ya cifrado en sesiones pasadas permanezca protegido.
- Firmar el intercambio Diffie-Hellman (o los parámetros y claves públicas involucradas) con las llaves de autenticación de RSA/certificados descritas arriba, para eliminar el riesgo de ataque de intermediario que tiene DH por sí solo.
- Usar AES (idealmente en modo autenticado, como AES-GCM) para cifrar el tráfico masivo entre estaciones y microservicios, aprovechando su alto rendimiento.

Este es, en esencia, el mismo principio de diseño que utilizan protocolos ya probados en producción como TLS 1.3 y SSH: autenticación asimétrica + acuerdo de clave efímero por Diffie-Hellman + cifrado simétrico de los datos. Adoptarlo permitiría a la organización eliminar por completo la práctica riesgosa observada en el segmento OTHER de compartir claves simétricas manualmente por canales no autenticados, sin sacrificar el rendimiento que exige el volumen de datos de la nueva plataforma analítica.

6. Respuestas a las preguntas

7. Referencias